
AVAX Vault Audit

AlwaysBeGrowing

Reviewer

Kyle Trusler | Namaskar

December 2, 2024

1 Executive Summary

Over the course of one week, [SeaFi](#) engaged with [Always Be Growing](#) to review [AVAX Vault](#).

A total of 3 issues were found.

Repository	Commit
AVAX Vault	13c844c413984c23c7dca2332467280c3620fbfd

Summary

Type of Project	Smart Contract
Timeline	Nov 25, 2024 - Dec 2, 2024
Methods	Manual Review
Documentation	Medium
Testing Coverage	Full Coverage

Total Issues

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	2
Gas Optimizations and Informational	1

Contents

1	Executive Summary	1
2	Always Be Growing	3
3	Introduction	3
4	Architecture	3
5	Methodology	4
6	Findings	4
6.1	Critical Risk	4
6.2	High Risk	4
6.3	Medium Risk	4
6.4	Low Risk	5
6.4.1	Potential Overflow in Reward Calculations	5
6.4.2	Centralization Risks in Node Operation	5
6.5	Informational	6
6.5.1	Shadowing Variables	6
6.5.2	Centralization Risks in Node Operation	7
6.6	Conclusion	7
7	Additional Risks	8
7.1	AVAX Price fluctuations	8
7.2	Upgradable Contracts	8
7.2.1	AVAX Vault	8
7.2.2	Avalanche Network Contracts	8
7.3	External Contract Interaction	8
7.3.1	Native AVAX Handling	8
7.3.2	Staking Contract	9
7.4	Centralization Risk	9
7.4.1	Elevated Privileges	9
7.5	Other Misc	10

2 Always Be Growing

ABG is a talented group of engineers focused on growing the web3 ecosystem. Learn more at <https://abg.garden>

3 Introduction

The following is a security review of the AVAX Vault contract.

The AVAX Vault contract is an [ERC4626](#) compliant vault that accepts native AVAX as the underlying asset. The funds are utilized in a strategy of staking AVAX tokens on behalf of trusted node operators and receiving the rewards generated as yield.

Upon deposit of AVAX, a number of ERC20 receipt tokens of xAVAX will be minted. These tokens represent the share of assets held within the contract (referred to henceforth as the "vault"). The vault will use the AVAX tokens to generate yield. The strategy for generating yield involves staking the AVAX on behalf of trusted node operators participating in the Avalanche network. The Avalanche network provides staking rewards, and those will be accrued by the vault, and increase the value of the xAVAX tokens. The xAVAX tokens can be redeemed for the underlying AVAX.

Disclaimer: This security review does not guarantee against a hack. It is a snapshot in time of brick according to the specific commit by a single human. Any modifications to the code will require a new security review.

4 Architecture

The vault inherits much of its base functionality from trusted and audited contracts from the [OpenZeppelin contract library](#). However, the vault has been modified to incorporate features for added security and the ability to interact with the Avalanche network. The vault has been designed to be upgradable, allowing for future improvements and bug fixes.

The areas of focus therefore are:

1. Does the contract adhere to the ERC-4626 standard?
2. Do the security features work as intended?

3. Are the changes to the OpenZeppelin contracts secure?

5 Methodology

The following are steps I took to evaluate the system:

- Clone & Setup project
- Read Readme and related documentation
- Line by line review
 - Contract
 - Interface
 - Unit Tests
 - Integration Tests
- Review Deployment Strategy
- Review Operation Strategy
- Run [Slither](#)
- Run ERC-4626 Solmate Test Suite

6 Findings

6.1 Critical Risk

No findings

6.2 High Risk

No findings

6.3 Medium Risk

No findings

6.4 Low Risk

3 findings

6.4.1 Potential Overflow in Reward Calculations

Severity: Low

Context: `AVAXVault.sol#getPendingRewards`

Description: The reward calculation in `getPendingRewards()` could potentially overflow with extremely large values:

```
uint256 newRewards = (totalAssets() * targetAPR * timeElapsed) / (10000 * 365
↳ days);
```

While unlikely in practice due to the total supply limitations of AVAX and reasonable APR values, this represents a theoretical edge case that could affect reward distributions if the multiplication of these values exceeds `uint256`.

Recommendation: Consider adding checks for large values or using a different calculation order to prevent potential overflow:

```
uint256 newRewards = (totalAssets() * targetAPR) / 10000;
newRewards = (newRewards * timeElapsed) / (365 days);
```

6.4.2 Centralization Risks in Node Operation

Severity: Informational

Context: `AVAXVault.sol`

Description: The vault's design relies heavily on trusted node operators and administrative roles. This creates several centralization points:

1. Node operators have direct control over staking operations and receive direct token transfers
2. Admin roles can modify critical parameters (`AVAXCap`, `targetAPR`)
3. The upgrade mechanism is controlled by a single owner
4. Node operators receive direct token transfers through `_stakeOnNode`, requiring a high level of trust

Recommendation: Consider implementing:

- Multi-signature requirements for critical operations
- Time-locks for parameter changes
- Gradual role transition mechanisms
- Documentation clearly explaining the trust model to users
- Additional safeguards around direct token transfers to node operators

6.5 Informational

2 findings

6.5.1 Shadowing Variables

Severity: Informational

Context: [AVAXVault.sol#L140](#)

Description:

The vault inherits from the `OwnableUpgradeable` contract, which has a function `owner`. It is good practice to avoid using the same name for newly declared variables, as it can lead to confusion and unexpected behavior. This is not a security risk for the vault with this particular code, but it can be confusing to developers and may lead to unexpected behavior.

```
function maxRedeem(address owner) public view override returns (uint256) {  
    // `owner` here shadows the `owner` function from OwnableUpgradeable  
}
```

Recommendation: Rename the variable to something that does not shadow the `owner` variable from `OwnableUpgradeable`. I recommend `shareHolder` as it is more descriptive of the variable's purpose.

```
- function maxRedeem(address owner)  
+ function maxRedeem(address shareHolder)
```

Resolution The instances of the `owner` variable have been changed to `shareHolder` in commit [cc704a5](#)

6.5.2 Centralization Risks in Node Operation

Severity: Informational

Context: [AVAXVault.sol](#)

Description: The vault's design relies heavily on trusted node operators and administrative roles. This creates several centralization points:

1. Node operators have direct control over staking operations and receive direct token transfers
2. Admin roles can modify critical parameters (AVAXCap, targetAPR)
3. The upgrade mechanism is controlled by a single owner
4. Node operators receive direct token transfers through `_stakeOnNode`, requiring a high level of trust

Recommendation: Consider implementing:

- Multi-signature requirements for critical operations
- Time-locks for parameter changes
- Gradual role transition mechanisms
- Documentation clearly explaining the trust model to users
- Additional safeguards around direct token transfers to node operators

6.6 Conclusion

The findings were of low and informational severity. The low-severity findings were a potential griefing in the implementation contract and the unbound variable setting. The informational severity finding was a shadowed variable. There are additional risks to consider, such as the AVAX price fluctuations, the upgradability of the contracts, and the centralization risk.

I believe the answers to the questions set out to be: The contract adheres to the ERC-4626 standard, the security features work as intended, and the changes to the OpenZeppelin contracts are secure.

7 Additional Risks

There are some additional risks. While they are not directly related to the code, they are still important to consider.

7.1 AVAX Price fluctuations

Given the volatile price of AVAX, the vault will ultimately be exposed to those price changes. The direct consequence of these changes can be seen in the node's collateralization ratio. If the price drops significantly, the node will be under collateralized and will receive reduced rewards, however the AVAX Cap will be artificially low, and overall have reward inefficiency. On the other hand, if the price increases significantly, the node will be overly collateralized, the AVAX Cap will be too high, and it will accrue more in rewards than in actuality. This delta could potentially lead to insufficient liquidity in the vault.

7.2 Upgradable Contracts

7.2.1 AVAX Vault

The vault itself is upgradable. This introduces a risk of the contract changing at any time by the owner. Additionally, there is added complexity, which could lead to a faulty upgrade. This process can be mitigated by having a robust upgrade process, including a well-tested contract before the upgrade.

7.2.2 Avalanche Network Contracts

This contract interacts with the Avalanche network's staking mechanism. The Avalanche network itself is upgradable. This introduces a risk of the contract changing at any time by the owner. As a result, the contracts that are being interacted with: the storage and the staking contracts, should be monitored by the team for changes.

7.3 External Contract Interaction

7.3.1 Native AVAX Handling

During normal operations of the vault, the transfers of native AVAX are done through WAVAX wrapping/unwrapping in the `depositAVAX()` and `redeemAVAX()`

functions. While the contract properly handles these operations and includes appropriate `receive()` function restrictions, the complexity of handling both native AVAX and WAVAX increases the potential attack surface.

7.3.2 Staking Contract

The vault interacts with the Avalanche network's staking mechanism.

In `AVAXVault.stakeAndDistributeRewards`, the external function calls `interact` with Avalanche's staking contract. This is a trusted contract without a risk of re-entry. The staking mechanics are governed by the Avalanche network protocol itself, which provides a stable and secure foundation for the vault's operations.

7.4 Centralization Risk

7.4.1 Elevated Privileges

The vault operates with a significant degree of centralization through several key roles:

1. Node Operators:

- Receive direct token transfers
- Control staking operations
- Can influence reward distribution timing

2. Contract Owner:

- Can upgrade the contract
- Controls `AVAXCap` and `targetAPR` parameters
- Manages node operator roles

This centralized control structure introduces risks:

- Single points of failure in key operations
- Potential for malicious actions if privileged accounts are compromised
- Direct token transfers to node operators require significant trust

The team should consider implementing additional checks and balances:

- Timelock mechanisms for parameter changes

- Multi-signature requirements for critical operations
- Transparent communication about operator selection and monitoring

7.5 Other Misc

The Solidity version ^0.8.20 used in the vault is potentially too new to be fully trusted.

Test coverage is complete.

The contract is documented with NatSpec.